

আপনি বলেছেন:

SQL query practice lab . assume sir data dibe blbe table bnaw then tar upr boro boro query dibe .erkm practice dw .DBMS exam jonno

ChatGPT বলেছে:

একদম ঠিক ধরেছো 🍌 — DBMS exam এ ঠিক এই ধরনের SQL Query Lab Practice দেওয়া হয় —
মানে:

- 1] প্রথমে sir একটা table structure দিবে (যেমন `student`, `department`, `sales`, ইত্যাদি)।
- 2] তারপর সেই টেবিলের উপর ভিত্তি করে complex query লিখতে বলবে — যেমন JOIN, GROUP BY, aggregate function, subquery, ইত্যাদি।

চলো আমি তোমাকে ঠিক ওইভাবে practice lab সেট করে দিচ্ছি 🙌



SQL PRACTICE LAB (for DBMS Exam)

◆ Table Creation Section

1] Create `STUDENT` Table

```
CREATE TABLE Student ( student_id INT PRIMARY KEY, name VARCHAR(50),  
department VARCHAR(30), cgpa DECIMAL(3,2), age INT );
```

2] Insert Sample Data

```
INSERT INTO Student VALUES (101, 'Maria', 'CSE', 3.85, 22), (102, 'Sajib',  
'EEE', 3.60, 23), (103, 'Rahim', 'CSE', 3.20, 21), (104, 'Karim', 'BBA',  
3.50, 22), (105, 'Nusrat', 'CSE', 3.90, 24);
```

◆ Table 2: DEPARTMENT

```
CREATE TABLE Department ( dept_id INT PRIMARY KEY, dept_name VARCHAR(30),  
total_students INT ); INSERT INTO Department VALUES (1, 'CSE', 3), (2,  
'EEE', 1), (3, 'BBA', 1);
```

♦ PRACTICE QUERIES (Exam Style)

🧩 Basic SELECT Queries

1. Show all student names and their departments.

```
SELECT          FROM
```

3. Show students who belong to 'CSE' department.

```
SELECT  FROM          WHERE  'CSE'
```

5. Display names and CGPA of students whose CGPA > 3.5.

```
SELECT          FROM          WHERE  3.5
```

🧩 Aggregate & GROUP BY

4. Find the average CGPA of each department.

```
SELECT          AVG  AS          FROM          GROUP BY
```

6. Show department having more than 2 students.

```
SELECT          COUNT  AS          FROM          GROUP BY  
HAVING COUNT  2
```

🧩 JOIN Queries

6. Display each student's name along with their department's total students.

```
SELECT          FROM          JOIN  
ON
```

8. List all departments and their students (even if no students exist).

```
SELECT          FROM          LEFT JOIN          ON
```

🧩 Subquery Practice

8. Find the student(s) having the highest CGPA.

SELECT FROM WHERE SELECT MAX FROM

10. Find students whose CGPA is greater than average CGPA.

SELECT FROM WHERE SELECT AVG FROM

Update, Delete, Alter Practice

10. Update Nusrat's CGPA to 4.00

UPDATE SET 4.00 WHERE 'Nusrat'

12. Delete the student whose department is 'BBA'.

DELETE FROM WHERE 'BBA'

14. Add a new column `email` in Student table.

ALTER TABLE ADD VARCHAR 50

Advanced Exam Level

13. Find the department with maximum average CGPA.

SELECT FROM GROUP BY ORDER BY
AVG DESC 1

15. Find all students older than the youngest CSE student.

SELECT FROM WHERE SELECT MIN FROM
WHERE 'CSE'

SELECT CASE WHEN 3.8 THEN 'Topper' WHEN
BETWEEN 3.0 AND 3.79 THEN 'Average' ELSE 'Needs Improvement'
END AS FROM

Bonus Practice Ideas (Sir Type Questions)

Teacher

Course

চাওলে আমি এর পরের ল্যাভে `SALES`, `EMPLOYEE`, `CUSTOMER` টেবিল নিয়েও exam-level practice set বানিয়ে দিতে পারি (with 15 complex queries: JOIN, nested, date functions, etc.)

তুমি কি পরের lab টা চাও — "SALES MANAGEMENT DATABASE Practice Set"?

আপনি বলেছেন:

Complex practice dw boro boro . subquery join multiple.advance practice

ChatGPT বলেছে:

চমৎকার 🔥 তুমি এখন Advanced SQL Exam-level Query Practice চাও — যেখানে থাকবে:

- ✓ multiple JOIN
- ✓ nested (multi-level) subquery
- ✓ aggregate with GROUP BY + HAVING
- ✓ advanced filtering
- ✓ real-world style questions

চলো, নিচে আমি পুরো একটা "SALES MANAGEMENT DATABASE" সেট তৈরি করে দিচ্ছি 👉
(এইটা DBMS exam এ খুব common type — full marks er question এখান থেকে আসে)

ADVANCED SQL PRACTICE LAB – SALES MANAGEMENT DATABASE

TABLE CREATION

1] Customer Table

```
CREATE TABLE Customer ( cust_id INT PRIMARY KEY, cust_name VARCHAR(50),  
city VARCHAR(40), grade INT );
```

2 Salesman Table

```
CREATE TABLE Salesman ( salesman_id INT PRIMARY KEY, name VARCHAR(50),  
city VARCHAR(40), commission DECIMAL(4,2) );
```

3 Orders Table

```
CREATE TABLE Orders ( order_id INT PRIMARY KEY, order_date DATE, cust_id  
INT, salesman_id INT, amount DECIMAL(10,2), FOREIGN KEY (cust_id)  
REFERENCES Customer(cust_id), FOREIGN KEY (salesman_id) REFERENCES  
Salesman(salesman_id) );
```



Sample Data

```
INSERT INTO Customer VALUES (101, 'Maria', 'Dhaka', 200), (102, 'Sajib',  
'Chittagong', 300), (103, 'Karim', 'Dhaka', 100), (104, 'Nusrat',  
'Khulna', 400), (105, 'Rahim', 'Rajshahi', 200); INSERT INTO Salesman  
VALUES (201, 'Imran', 'Dhaka', 0.15), (202, 'Hasan', 'Chittagong', 0.12),  
(203, 'Fahim', 'Khulna', 0.10), (204, 'Sabbir', 'Dhaka', 0.18); INSERT  
INTO Orders VALUES (301, '2024-01-15', 101, 201, 5000), (302,  
'2024-02-10', 102, 202, 2500), (303, '2024-03-05', 103, 201, 1000), (304,  
'2024-04-20', 104, 203, 8000), (305, '2024-05-25', 105, 204, 6000), (306,  
'2024-06-10', 101, 201, 7000), (307, '2024-07-15', 102, 204, 4000);
```

⚡ COMPLEX QUERY PRACTICE

- ◆ 1. Find customers who placed orders more than once.

```
SELECT cust_id, COUNT(order_id) AS total_orders FROM Orders GROUP BY  
cust_id HAVING COUNT(order_id) > 1;
```

♦ **2. Display each customer's name, salesman name, and order amount.**

```
SELECT c.cust_name, s.name AS salesman_name, o.amount FROM Orders o JOIN
Customer c ON o.cust_id = c.cust_id JOIN Salesman s ON o.salesman_id =
s.salesman_id;
```

♦ **3. Find customers who ordered from a salesman of the same city.**

```
SELECT DISTINCT c.cust_name, s.name, c.city FROM Customer c JOIN Salesman
s ON c.city = s.city;
```

♦ **4. Show salesman name and total sales handled by each.**

```
SELECT s.name AS salesman_name, SUM(o.amount) AS total_sales FROM Orders o
JOIN Salesman s ON o.salesman_id = s.salesman_id GROUP BY s.name ORDER BY
total_sales DESC;
```

♦ **5. Find salesman who sold more than 10000 total in orders.**

```
SELECT s.name, SUM(o.amount) AS total_sales FROM Salesman s JOIN Orders o
ON s.salesman_id = o.salesman_id GROUP BY s.name HAVING SUM(o.amount) >
10000;
```

♦ **6. Find customer(s) who spent more than the average order amount.**

```
SELECT c.cust_name, SUM(o.amount) AS total_spent FROM Customer c JOIN
Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_name HAVING
SUM(o.amount) > ( SELECT AVG(amount) FROM Orders );
```

♦ **7. Find customers served by the salesman with the highest commission.**

```
SELECT DISTINCT c.cust_name FROM Customer c JOIN Orders o ON c.cust_id =  
o.cust_id JOIN Salesman s ON o.salesman_id = s.salesman_id WHERE  
s.commission = (SELECT MAX(commission) FROM Salesman);
```

♦ **8. Display salesman name, their total number of customers, and total order value.**

```
SELECT s.name, COUNT(DISTINCT o.cust_id) AS total_customers, SUM(o.amount)  
AS total_sales FROM Salesman s JOIN Orders o ON s.salesman_id =  
o.salesman_id GROUP BY s.name;
```

♦ **9. Find the city that generated the highest total order amount.**

```
SELECT c.city, SUM(o.amount) AS total_amount FROM Orders o JOIN Customer c  
ON o.cust_id = c.cust_id GROUP BY c.city ORDER BY total_amount DESC LIMIT  
1;
```

♦ **10. Find customer who placed order handled by salesman from a different city.**

```
SELECT DISTINCT c.cust_name, s.name AS salesman_name, c.city AS  
customer_city, s.city AS salesman_city FROM Orders o JOIN Customer c ON  
o.cust_id = c.cust_id JOIN Salesman s ON o.salesman_id = s.salesman_id  
WHERE c.city <> s.city;
```

♦ **11. Find orders whose amount is greater than the average of their salesman's all orders.**

```
SELECT o.order_id, c.cust_name, s.name, o.amount FROM Orders o JOIN  
Salesman s ON o.salesman_id = s.salesman_id JOIN Customer c ON o.cust_id =  
c.cust_id WHERE o.amount > ( SELECT AVG(o2.amount) FROM Orders o2 WHERE  
o2.salesman_id = s.salesman_id );
```

♦ **12. Show salesman who didn't handle any order.**

```
SELECT s.name FROM Salesman s LEFT JOIN Orders o ON s.salesman_id =  
o.salesman_id WHERE o.order_id IS NULL;
```

♦ **13. Find the 2nd highest order amount and who placed it.**

```
SELECT c.cust_name, o.amount FROM Orders o JOIN Customer c ON o.cust_id =  
c.cust_id WHERE o.amount = ( SELECT MAX(amount) FROM Orders WHERE amount <  
(SELECT MAX(amount) FROM Orders) );
```

♦ **14. Find salesman who handled orders from at least 2 different cities.**

```
SELECT s.name FROM Salesman s JOIN Orders o ON s.salesman_id =  
o.salesman_id JOIN Customer c ON o.cust_id = c.cust_id GROUP BY s.name  
HAVING COUNT(DISTINCT c.city) >= 2;
```

♦ **15. Display each customer, their total order, and grade label:**

```
SELECT c.cust_name, c.grade, CASE WHEN c.grade < 200 THEN 'Low' WHEN  
c.grade BETWEEN 200 AND 300 THEN 'Medium' ELSE 'High' END AS Grade_Level  
FROM Customer c;
```

 BONUS (Nested + Join Combo)

♦ **16. Find salesman who handled the largest single order in each city.**

```
SELECT s.name, c.city, o.amount FROM Orders o JOIN Customer c ON o.cust_id  
= c.cust_id JOIN Salesman s ON o.salesman_id = s.salesman_id WHERE
```



```
o.amount = ( SELECT MAX(o2.amount) FROM Orders o2 JOIN Customer c2 ON  
o2.cust_id = c2.cust_id WHERE c2.city = c.city );
```

✓ Ready for Exam

এই সেটটা প্র্যাকটিস করলে তুমি পারবে:

চাওলে আমি এর পরের “University Database Advanced Query Set

(Student–Course–Faculty)” তৈরি করে দিতে পারি — ওটা university type query নিয়ে (exam এ খুব common)।

👉 কি সেইটা বানিয়ে দেই?

আপনি বলেছেন:

Amra akta shop niye project krsilm oi quality boror boro practice dw real life jto rkm hy

ChatGPT বলেছে:

একদম পারফেক্ট চিন্তা করেছো 🔥

যেহেতু তোমরা “Shop Management / Discount Shop” প্রজেক্ট করেছো (যেমন তোমার Taqwa Discount Shop DBMS Project),

তাহলে ঠিক ওইরকম বাস্তবধর্মী (real-life) database structure নিয়ে বড় বড় SQL query practice করলে — exam এ 100% confident হবে 💪

চলো, নিচে আমি দিচ্ছি পুরো Real-Life Advanced SQL Lab: Shop Management System Practice Set

👉 এখানে থাকবে realistic tables, data, এবং complex 18+ queries (JOIN, nested subquery, analytic thinking)।



TAQWA DISCOUNT SHOP – ADVANCED SQL PRACTICE LAB



TABLE CREATION

1) Product Table

```
CREATE TABLE Product ( product_id INT PRIMARY KEY, product_name  
VARCHAR(50), category VARCHAR(30), price DECIMAL(10,2), stock INT );
```

2) Customer Table

```
CREATE TABLE Customer ( cust_id INT PRIMARY KEY, cust_name VARCHAR(50),  
city VARCHAR(40), phone VARCHAR(15) );
```

3) Orders Table

```
CREATE TABLE Orders ( order_id INT PRIMARY KEY, cust_id INT, order_date  
DATE, total_amount DECIMAL(10,2), FOREIGN KEY (cust_id) REFERENCES  
Customer(cust_id) );
```

4) OrderDetails Table

```
CREATE TABLE OrderDetails ( orderdetail_id INT PRIMARY KEY, order_id INT,  
product_id INT, quantity INT, discount DECIMAL(4,2), FOREIGN KEY  
(order_id) REFERENCES Orders(order_id), FOREIGN KEY (product_id)  
REFERENCES Product(product_id) );
```

5) Employee Table

```
CREATE TABLE Employee ( emp_id INT PRIMARY KEY, emp_name VARCHAR(50), role  
VARCHAR(30), salary DECIMAL(10,2), city VARCHAR(40) );
```



SAMPLE DATA

```
INSERT INTO Product VALUES (1, 'Rice 5kg', 'Grocery', 500, 100), (2,  
'Sugar 1kg', 'Grocery', 90, 50), (3, 'Oil 2L', 'Grocery', 300, 80), (4,  
'Shampoo 200ml', 'Cosmetics', 150, 40), (5, 'Soap', 'Cosmetics', 60, 200),  
(6, 'Laptop', 'Electronics', 70000, 10); INSERT INTO Customer VALUES (101,
```

```
'Maria', 'Dhaka', '01700000001'), (102, 'Sajib', 'Khulna', '01700000002'),
(103, 'Nusrat', 'Dhaka', '01700000003'), (104, 'Karim', 'Rajshahi',
'01700000004'); INSERT INTO Orders VALUES (201, 101, '2024-01-10', 590),
(202, 102, '2024-02-15', 320), (203, 103, '2024-03-01', 70500), (204, 101,
'2024-03-20', 180), (205, 104, '2024-04-10', 120); INSERT INTO
OrderDetails VALUES (301, 201, 1, 1, 0.05), (302, 201, 2, 1, 0.10), (303,
202, 5, 2, 0.00), (304, 203, 6, 1, 0.15), (305, 204, 4, 1, 0.05), (306,
205, 2, 1, 0.00); INSERT INTO Employee VALUES (401, 'Imran', 'Manager',
40000, 'Dhaka'), (402, 'Hasan', 'Salesman', 25000, 'Khulna'), (403,
'Nafis', 'Cashier', 22000, 'Dhaka'), (404, 'Sadia', 'Sales Assistant',
20000, 'Rajshahi');
```

🌟 REAL-LIFE ADVANCED SQL QUERIES

- ◆ 1. Show total number of customers from each city.

```
SELECT city, COUNT(cust_id) AS total_customers FROM Customer GROUP BY
city;
```

-
- ◆ 2. Find the total stock value (price × quantity) for each product.

```
SELECT product_name, (price * stock) AS total_stock_value FROM Product
ORDER BY total_stock_value DESC;
```

-
- ◆ 3. Display all orders with customer name and order total.

```
SELECT o.order_id, c.cust_name, o.order_date, o.total_amount FROM Orders o
JOIN Customer c ON o.cust_id = c.cust_id;
```

-
- ◆ 4. Find which customer has placed the highest total order amount overall.

```
SELECT c.cust_name, SUM(o.total_amount) AS total_spent FROM Orders o JOIN
Customer c ON o.cust_id = c.cust_id GROUP BY c.cust_name ORDER BY
total_spent DESC LIMIT 1;
```

♦ **5. Find the most expensive product ordered.**

```
SELECT p.product_name, p.price FROM Product p WHERE p.price = (SELECT
MAX(price) FROM Product);
```

♦ **6. Show all orders where any product got discount > 10%.**

```
SELECT DISTINCT o.order_id, c.cust_name, od.discount FROM Orders o JOIN
OrderDetails od ON o.order_id = od.order_id JOIN Customer c ON o.cust_id =
c.cust_id WHERE od.discount > 0.10;
```

♦ **7. Find the average discount given on each category of product.**

```
SELECT p.category, AVG(od.discount) AS avg_discount FROM OrderDetails od
JOIN Product p ON od.product_id = p.product_id GROUP BY p.category;
```

♦ **8. Show product(s) that are out of stock or below 10 in stock.**

```
SELECT product_name, stock FROM Product WHERE stock <= 10;
```

♦ **9. Find total sales amount for each category (join 3 tables).**

```
SELECT p.category, SUM(od.quantity * p.price * (1 - od.discount)) AS
total_sales FROM Product p JOIN OrderDetails od ON p.product_id =
od.product_id JOIN Orders o ON od.order_id = o.order_id GROUP BY
p.category;
```

♦ **10. Find customers who bought products from more than one category.**

```
SELECT c.cust_name, COUNT(DISTINCT p.category) AS category_count FROM
Customer c JOIN Orders o ON c.cust_id = o.cust_id JOIN OrderDetails od ON
o.order_id = od.order_id JOIN Product p ON od.product_id = p.product_id
GROUP BY c.cust_name HAVING COUNT(DISTINCT p.category) > 1;
```

♦ **11. Find customers who never ordered anything.**

```
SELECT c.cust_name FROM Customer c LEFT JOIN Orders o ON c.cust_id =
o.cust_id WHERE o.order_id IS NULL;
```

♦ **12. Find employees who work in the same city as any customer.**

```
SELECT DISTINCT e.emp_name, e.city FROM Employee e JOIN Customer c ON
e.city = c.city;
```

♦ **13. Find which employee earns more than the average salary.**

```
SELECT emp_name, salary FROM Employee WHERE salary > (SELECT AVG(salary)
FROM Employee);
```

♦ **14. Find top 3 most sold products.**

```
SELECT p.product_name, SUM(od.quantity) AS total_sold FROM OrderDetails od
JOIN Product p ON od.product_id = p.product_id GROUP BY p.product_name
ORDER BY total_sold DESC LIMIT 3;
```

♦ **15. Find which product generated the highest total revenue.**

```
SELECT p.product_name, SUM(od.quantity * p.price) AS revenue FROM Product
p JOIN OrderDetails od ON p.product_id = od.product_id GROUP BY
p.product_name ORDER BY revenue DESC LIMIT 1;
```

♦ **16. Find all customers who ordered in March 2024.**

```
SELECT c.cust_name, o.order_id, o.order_date FROM Orders o JOIN Customer c
ON o.cust_id = c.cust_id WHERE MONTH(o.order_date) = 3 AND
YEAR(o.order_date) = 2024;
```

◆ 17. Find which city has the highest total order amount.

```
SELECT c.city, SUM(o.total_amount) AS total_sales FROM Orders o JOIN
Customer c ON o.cust_id = c.cust_id GROUP BY c.city ORDER BY total_sales
DESC LIMIT 1;
```

◆ 18. Nested Subquery – Find product(s) more expensive than the average price of Electronics category.

```
SELECT product_name, price FROM Product WHERE price > ( SELECT AVG(price)
FROM Product WHERE category = 'Electronics' );
```

Bonus Challenge (Exam “Case Study” Style)

19 Find the total monthly sales report (month, total sales, number of orders).

```
SELECT MONTH(order_date) AS month, SUM(total_amount) AS total_sales,
COUNT(order_id) AS total_orders FROM Orders GROUP BY MONTH(order_date)
ORDER BY month;
```

RESULT

এই ল্যাবটা করলে তুমি শিখবে—

- ✓ Multiple Table Join (3–4 table একসাথে)
- ✓ Nested Subquery
- ✓ Aggregation + HAVING + Conditional Filter
- ✓ Business analytics type question (e.g., top customer, monthly report, etc.)
- ✓ Real database thinking

চাওলে আমি এই Shop Database এর উপর “Mini SQL Exam Paper” (10 questions, mixed difficulty, marks সহ) তৈরি করে দিতে পারি —

যেটা তুমি solve করে নিজের exam readiness check করতে পারবে ✅

👉 কি আমি ওই mini exam paper টা বানিয়ে দিই?

আপনি বলেছেন:

R Kono type bakinase?

ChatGPT বলেছে:

চমৎকার প্রশ্ন! 💡

তুমি এখন পর্যন্ত তিনটা বড় type এর SQL practice সেট দেখেছো —

- 👉 Student–Department (Academic)
- 👉 Sales Management System (Business)
- 👉 Shop Management System (Retail/Commercial)

কিন্তু বাস্তবে ও exam এ আরো কিছু “ধরনভেদ” বা real-world database type থাকে, যেগুলো দিয়ে অনেক complex SQL question দেওয়া হয়।

চলো আমি পুরো লিস্টটা ব্যাখ্যা করি 👉

🕒 All Major Types of SQL Practice / Project Database (Exam + Real Life Use)

🏠 1 University / Academic Management System

Tables: Student, Course, Faculty, Enrollment, Department

Practice Focus:

Example Query:

```
SELECT s.name, c.course_name, e.grade FROM Student s JOIN Enrollment e ON  
s.stu_id = e.stu_id JOIN Course c ON e.course_id = c.course_id WHERE  
e.grade = ( SELECT MAX(grade) FROM Enrollment WHERE course_id =  
c.course_id );
```

2 Hospital Management System

Tables: Doctor, Patient, Appointment, Prescription, Department

Practice Focus:

Example:

```
SELECT d.name, COUNT(a.appointment_id) AS total_appointments FROM Doctor d  
JOIN Appointment a ON d.doc_id = a.doc_id GROUP BY d.name ORDER BY  
total_appointments DESC;
```

3 Shop / Retail / Discount System (You did this one)

Focus:

4 Banking / ATM Management System

Tables: Account, Customer, Transaction, Branch, Loan

Practice Focus:

Example:

```
SELECT c.name, a.account_no, SUM(t.amount) AS total_txn FROM Customer c
JOIN Account a ON c.cust_id = a.cust_id JOIN Transaction t ON a.account_no
= t.account_no GROUP BY c.name, a.account_no;
```

5 E-Commerce / Online Order System

Tables: Customer, Product, Cart, Orders, Payment

Practice Focus:

Example:

```
SELECT c.name, SUM(o.total_amount) AS total_spent FROM Customer c JOIN
Orders o ON c.cust_id = o.cust_id GROUP BY c.name HAVING COUNT(o.order_id)
> 2;
```

6 Hotel / Booking Management

Tables: Hotel, Room, Customer, Booking, Payment

Practice Focus:

Example:

```
SELECT h.name, COUNT(b.booking_id) AS total_bookings FROM Hotel h JOIN
Booking b ON h.hotel_id = b.hotel_id GROUP BY h.name;
```

7 Employee / Payroll System

Tables: Employee, Department, Salary, Attendance

Practice Focus:

Example:

```
SELECT e.name, e.salary, d.dept_name FROM Employee e JOIN Department d ON
e.dept_id = d.dept_id WHERE e.salary = ( SELECT MAX(salary) FROM Employee
WHERE dept_id = d.dept_id );
```

8 Transport / Bus Reservation

Tables: Bus, Driver, Route, Passenger, Booking

Practice Focus:

Example:

```
SELECT r.route_name, COUNT(b.booking_id) AS total_bookings FROM Route r
JOIN Booking b ON r.route_id = b.route_id GROUP BY r.route_name;
```

9 Inventory / Warehouse Management

Tables: Item, Supplier, Purchase, Stock, Category

Practice Focus:

Example:

```
SELECT s.supplier_name, SUM(p.quantity) AS total_supplied FROM Supplier s
JOIN Purchase p ON s.supplier_id = p.supplier_id GROUP BY s.supplier_name
ORDER BY total_supplied DESC;
```



10

Government / Citizen Management (Advanced DBMS Projects)

Tables: Citizen, TaxRecord, Property, District

Focus:



Summary Table of Practice Areas

চাওলে আমি এখন তোমার জন্য পরবর্তী Advanced Practice Lab বানিয়ে দিতে পারি —

তুমি বেছে নাও নিচের থেকে 

- 1] University Database (Advanced Queries)
- 2] Banking System (Nested + Transaction Focus)
- 3] E-Commerce Website Database (Real-World Analysis)
- 4] Employee Payroll System (Salary Analytics)

কোনটা দিয়ে পরের ল্যাব বানাবো?

আপনি বলেছেন:

); assa elhane id match check na dile ki hto

ChatGPT বলেছে:

Excellent question — and you're thinking like a real SQL analyst 🙌

Let's break this query step-by-step to understand why the ID match (`WHERE`

`o2.salesman_id = s.salesman_id`) is so important, and what would happen if you removed it.

Query recap

```
SELECT o.order_id, c.cust_name, s.name, o.amount FROM Orders o JOIN
Salesman s ON o.salesman_id = s.salesman_id JOIN Customer c ON o.cust_id =
c.cust_id WHERE o.amount > ( SELECT AVG(o2.amount) FROM Orders o2 WHERE
o2.salesman_id = s.salesman_id );
```

Meaning:

For each order `o`, this query checks if that order's `amount` is greater than the average order amount of the same salesman (`s.salesman_id`).

So the condition:

```
WHERE o2.salesman_id = s.salesman_id
```

links the outer query's salesman to the inner subquery's salesman — creating a correlated subquery.

If you remove this line:

```
WHERE o2.salesman_id = s.salesman_id
```

Then the query becomes:

```
WHERE o.amount > (SELECT AVG(o2.amount) FROM Orders o2);
```

Now, the subquery calculates one global average — the average order amount of *all* salesmen, not per individual salesman.

Example to illustrate:

✓ With the condition (`WHERE o2.salesman_id = s.salesman_id`):

Now query checks:



✗ Without the condition:

Now query checks:



Conclusion

✓ `WHERE o2.salesman_id =`
`s.salesman_id`



💡 Real-World Tip:

Always ensure subqueries in `WHERE` are properly correlated when comparing related groups (like per salesman, per customer, per region, etc.).

If you forget the `id` match, your query logic shifts from “group-based comparison” to “global comparison.”

Would you like me to give you 5 similar real-world subquery + join practice questions (increasing difficulty, with hints)?

These are great for DBMS exam prep.

আপনি বলেছেন:

`WHERE o.order_id IS NULL;` not in diye to hy

ChatGPT বলেছে:

✅ Excellent observation — yes, you’re absolutely right!

The query you wrote using `LEFT JOIN ... WHERE o.order_id IS NULL` can indeed be written using `NOT IN`.

But let’s carefully see how and why they are equivalent — and when they are not exactly the same.

🧠 Your Original Query

```
SELECT s.name FROM Salesman s LEFT JOIN Orders o ON s.salesman_id =  
o.salesman_id WHERE o.order_id IS NULL;
```

✅ Equivalent Query Using `NOT IN`

```
SELECT s.name FROM Salesman WHERE salesman_id NOT IN ( SELECT salesman_id  
FROM Orders );
```

👉 Meaning:

Both queries find salesmen who have not made any orders.

🧩 Step-by-step explanation

1] `LEFT JOIN + IS NULL` method:

`LEFT JOIN`

`NULL`

`WHERE o.order_id IS NULL`

✓ Output = Salesmen with no orders.

2] `NOT IN` subquery method:

`SELECT salesman_id FROM Orders`

✓ Output = Same — salesmen with no orders.

⚠ Important difference: NULL behavior

There's one big warning with `NOT IN`:

If the subquery returns even one NULL, the entire comparison fails (returns no rows).

Example:

Now:


```
WHERE salesman_id NOT IN (101, NULL)
```

✗ This returns no results, because SQL treats `NOT IN (NULL)` as unknown.

But:

```
LEFT JOIN ... WHERE o.order_id IS NULL
```

✓ Still works correctly — ignores NULL safely.



So summary:



`LEFT JOIN ... IS NULL`



`NOT IN (subquery)`



`NOT EXISTS (subquery)`



Best Practice:

If you want a safer and faster version (for real DBMS exam or industry use), use `NOT EXISTS`:

```
SELECT s.name FROM Salesman s WHERE NOT EXISTS ( SELECT 1 FROM Orders o
WHERE o.salesman_id = s.salesman_id );
```



Works perfectly



Safe with NULLs



Usually optimized better by database engines

Would you like me to give you a few practice problems comparing `NOT IN`, `NOT EXISTS`, and `LEFT JOIN ... IS NULL` with real-life shop data examples? It'll help you master all 3 methods for your DBMS exam.

আপনি বলেছেন:



); easy vbe

ChatGPT বলেছে:

Perfect 🍌 you picked one of the most important real-life SQL interview/exam patterns

“Find the top item per group” (here: the largest order per city).

Let's rewrite and explain this step-by-step in easy Bengali + English mix, so it's super clear.

Problem:

প্রতিটি শহরের (city) মধ্যে সেই salesman-কে খুঁজো যিনি সবচেয়ে বড় order handle করেছে।

Simple version (easy explanation):

```
SELECT s.name, c.city, o.amount FROM Orders o JOIN Customer c ON o.cust_id = c.cust_id JOIN Salesman s ON o.salesman_id = s.salesman_id WHERE
```

```
o.amount = ( SELECT MAX(o2.amount) FROM Orders o2 JOIN Customer c2 ON  
o2.cust_id = c2.cust_id WHERE c2.city = c.city );
```

Step-by-step (simple English + logic):

1] Main query:



2] Subquery:

```
SELECT MAX(o2.amount) FROM Orders o2 JOIN Customer c2 ON o2.cust_id =  
c2.cust_id WHERE c2.city = c.city
```

3] WHERE condition:

```
WHERE o.amount = (that city's maximum amount)
```

So if Dhaka's max order = 5000,
it'll only keep the row(s) from Dhaka where `amount = 5000`.

 Example to visualize

✓ Subquery per city gives:

Then outer query picks those exact rows.

Result:

💡 Easier “Group By” alternative (modern DBMS supports it)

You can also get the same result with window function (if allowed in exam):

```
SELECT s.name, c.city, o.amount FROM Orders o JOIN Customer c ON o.cust_id
= c.cust_id JOIN Salesman s ON o.salesman_id = s.salesman_id WHERE
o.amount = ( SELECT MAX(o2.amount) FROM Orders o2 JOIN Customer c2 ON
o2.cust_id = c2.cust_id WHERE c2.city = c.city );
```

✓ (above = original nested version)

OR with window function (simpler if allowed):

```
SELECT name, city, amount FROM ( SELECT s.name, c.city, o.amount, RANK()
OVER (PARTITION BY c.city ORDER BY o.amount DESC) AS rnk FROM Orders o
JOIN Customer c ON o.cust_id = c.cust_id JOIN Salesman s ON o.salesman_id
= s.salesman_id ) t WHERE rnk = 1;
```

📍 In summary



Would you like me to give you 5 more “Top per group” real-life queries like this one? (e.g., top-paid employee per department, best-selling product per category, etc.) — perfect DBMS practice level.

আপনি বলেছেন:

na group by use kro

ChatGPT বলেছে:

Excellent 🍌 — now you want the same query (find salesman who handled the largest single order in each city) but using `GROUP BY` instead of a subquery inside `WHERE`.

Let's make it step-by-step, in easy Bangla + English mix, so you understand the logic perfectly.

Goal:

প্রতিটি city তে যে salesman সবচেয়ে বড় order amount handle করেছে — সেটা `GROUP BY` দিয়েই বের করবো।

Step 1: Find the largest order amount per city

First, we find maximum order amount for each city.

```
SELECT c.city, MAX(o.amount) AS max_amount FROM Orders o JOIN Customer c
ON o.cust_id = c.cust_id GROUP BY c.city;
```

 Output (example):

🧩 Step 2: Join it back to get full details (salesman, customer, etc.)

Now we join this result with the main tables to get which salesman handled that max order.

```
SELECT s.name AS salesman_name, c.city, o.amount FROM Orders o JOIN
Customer c ON o.cust_id = c.cust_id JOIN Salesman s ON o.salesman_id =
s.salesman_id JOIN ( SELECT c2.city, MAX(o2.amount) AS max_amount FROM
Orders o2 JOIN Customer c2 ON o2.cust_id = c2.cust_id GROUP BY c2.city )
AS city_max ON c.city = city_max.city AND o.amount = city_max.max_amount;
```

✅ Easy explanation:

1] The inner query (`city_max`) finds:

```
MAX(amount)
```

2] The outer query joins it back:

✅ Final Result:



Notes:

GROUP BY

Would you like me to give you the same type of GROUP BY practice for:

আপনি বলেছেন:



◆

◆

◆

◆

◆

◆

◆

◆

◆

◆

◆



ORDER BY month; egula bangla soho bujhaw

ChatGPT বলেছে:

দারুন! 🍌

তুমি একদম DBMS Exam-level SQL Practice Set নিয়ে এসেছো — এবং এগুলো পুরোপুরি real-life eCommerce / Shop Management System থেকে নেওয়া ধরনের প্রশ্ন।

এখন নিচে আমি প্রতিটা query খুব সহজভাবে বাংলায় ব্যাখ্যা করব, যেন তুমি বুঝতে পারো কেন এটা কাজ করে, আর কী বের করে।

◆ 4. Find which customer has placed the highest total order amount overall.

```
SELECT c.cust_name, SUM(o.total_amount) AS total_spent FROM Orders o JOIN  
Customer c ON o.cust_id = c.cust_id GROUP BY c.cust_name ORDER BY  
total_spent DESC LIMIT 1;
```

🧠 Meaning:

প্রতিটা customer কত টাকার order দিয়েছে, তার total বের করে।

তারপর ORDER BY DESC দিয়ে সবচেয়ে বেশি খরচ করা customer টা নেয়া হয় (LIMIT 1 মানে শুধুমাত্র প্রথম জন)।

✅ Result: সেই গ্রাহক যিনি মোট টাকায় সবচেয়ে বেশি order দিয়েছেন।

♦ 5. Find the most expensive product ordered.

```
SELECT p.product_name, p.price FROM Product p WHERE p.price = (SELECT  
MAX(price) FROM Product);
```

🧠 Meaning:

MAX(price) দেয় প্রোডাক্ট টেবিলের সবচেয়ে দামী প্রোডাক্টের দাম।

বাইরের query সেটার সাথে মেলে এমন product দেখায়।

✅ Result: সবচেয়ে দামী প্রোডাক্ট।

♦ 6. Show all orders where any product got discount > 10%.

```
SELECT DISTINCT o.order_id, c.cust_name, od.discount FROM Orders o JOIN  
OrderDetails od ON o.order_id = od.order_id JOIN Customer c ON o.cust_id =  
c.cust_id WHERE od.discount > 0.10;
```

🧠 Meaning:

যেসব order এ কোনো প্রোডাক্টে ১০% এর বেশি ছাড় (discount) দেওয়া হয়েছে,

তাদের order_id, গ্রাহকের নাম, discount দেখায়।

✅ Result: Discount দেওয়া orders।

♦ 7. Find the average discount given on each category of product.

```
SELECT p.category, AVG(od.discount) AS avg_discount FROM OrderDetails od  
JOIN Product p ON od.product_id = p.product_id GROUP BY p.category;
```

🧠 Meaning:

প্রত্যেক product category অনুযায়ী discount এর গড় (average) বের করে।

✅ Result: প্রতিটি category এর গড় discount rate।

♦ 8. Show product(s) that are out of stock or below 10 in stock.

```
SELECT product_name, stock FROM Product WHERE stock <= 10;
```

🧠 Meaning:

যেসব প্রোডাক্টের স্টক শেষ বা খুব কম (≤ 10), সেগুলো দেখায়।

✅ Result: কম স্টক থাকা বা স্টক-আউট প্রোডাক্ট লিস্ট।

♦ 9. Find total sales amount for each category (join 3 tables).

```
SELECT p.category, SUM(od.quantity * p.price * (1 - od.discount)) AS  
total_sales FROM Product p JOIN OrderDetails od ON p.product_id =  
od.product_id JOIN Orders o ON od.order_id = o.order_id GROUP BY  
p.category;
```

🧠 Meaning:

প্রত্যেক category অনুযায়ী মোট বিক্রির টাকা বের করে:

✅ Result: প্রতিটি category এর total sales।

♦ 10. Find customers who bought products from more than one category.

```
SELECT c.cust_name, COUNT(DISTINCT p.category) AS category_count FROM  
Customer c JOIN Orders o ON c.cust_id = o.cust_id JOIN OrderDetails od ON  
o.order_id = od.order_id JOIN Product p ON od.product_id = p.product_id  
GROUP BY c.cust_name HAVING COUNT(DISTINCT p.category) > 1;
```

🧠 Meaning:

যেসব customer একের বেশি category থেকে প্রোডাক্ট কিনেছে, তাদের দেখায়।

✓ Result: Multi-category buyer গ্রাহকগণ।

♦ 11. Find customers who never ordered anything.

```
SELECT c.cust_name FROM Customer c LEFT JOIN Orders o ON c.cust_id =  
o.cust_id WHERE o.order_id IS NULL;
```

🧠 Meaning:

Customer টেবিলের মধ্যে যাদের কোনো order মেলেনি (null),
মানে তারা কখনো order দেয়নি।

✓ Result: যারা এখনো কিছু কেনেনি।

♦ 12. Find employees who work in the same city as any customer.

```
SELECT DISTINCT e.emp_name, e.city FROM Employee e JOIN Customer c ON  
e.city = c.city;
```

🧠 Meaning:

যেসব employee এমন শহরে কাজ করে, যেখানে customer-ও আছে।

✓ Result: Customer এর মতো একই শহরের কর্মচারী।

♦ 13. Find which employee earns more than the average salary.

```
SELECT emp_name, salary FROM Employee WHERE salary > (SELECT AVG(salary)  
FROM Employee);
```

🧠 Meaning:

সকল কর্মচারীর গড় বেতনের চেয়ে বেশি আয় করা employee বের করা হয়।

✓ Result: Above-average salaried employees।

♦ 14. Find top 3 most sold products.

```
SELECT p.product_name, SUM(od.quantity) AS total_sold FROM OrderDetails od
JOIN Product p ON od.product_id = p.product_id GROUP BY p.product_name
ORDER BY total_sold DESC LIMIT 3;
```

🧠 Meaning:

যে প্রোডাক্টগুলো সবচেয়ে বেশি বিক্রি হয়েছে (quantity অনুযায়ী),
শীর্ষ ৩টা দেখায়।

✅ Result: Top 3 best-selling products।

♦ 15. Find which product generated the highest total revenue.

```
SELECT p.product_name, SUM(od.quantity * p.price) AS revenue FROM Product
p JOIN OrderDetails od ON p.product_id = od.product_id GROUP BY
p.product_name ORDER BY revenue DESC LIMIT 1;
```

🧠 Meaning:

প্রত্যেক প্রোডাক্ট থেকে মোট বিক্রির টাকা (quantity × price) বের করে,
সবচেয়ে বেশি টাকার প্রোডাক্টটা নেয়া হয়।

✅ Result: Highest revenue product।

♦ 16. Find all customers who ordered in March 2024.

```
SELECT c.cust_name, o.order_id, o.order_date FROM Orders o JOIN Customer c
ON o.cust_id = c.cust_id WHERE MONTH(o.order_date) = 3 AND
YEAR(o.order_date) = 2024;
```

🧠 Meaning:

২০২৪ সালের মার্চ মাসে যারা order দিয়েছে তাদের লিস্ট।

✓ Result: March 2024 orders + customers |

♦ 17. Find which city has the highest total order amount.

```
SELECT c.city, SUM(o.total_amount) AS total_sales FROM Orders o JOIN  
Customer c ON o.cust_id = c.cust_id GROUP BY c.city ORDER BY total_sales  
DESC LIMIT 1;
```

🧠 Meaning:

প্রতিটি শহরের মোট order amount যোগ করে,
যে শহরে সবচেয়ে বেশি order হয়েছে সেটি দেখায়।

✓ Result: Highest sales city |

♦ 18. Product(s) more expensive than average price of Electronics.

```
SELECT product_name, price FROM Product WHERE price > ( SELECT AVG(price)  
FROM Product WHERE category = 'Electronics' );
```

🧠 Meaning:

Electronics ক্যাটাগরির গড় দামের চেয়ে বেশি দামী সব প্রোডাক্ট দেখায়।

✓ Result: Premium products |

🎯 19 Bonus: Monthly Sales Report

```
SELECT MONTH(order_date) AS month, SUM(total_amount) AS total_sales,  
COUNT(order_id) AS total_orders FROM Orders GROUP BY MONTH(order_date)  
ORDER BY month;
```

🧠 Meaning:

প্রতিমাসে মোট কত টাকা বিক্রি হয়েছে (SUM),

কতগুলো order হয়েছে (COUNT),

তা দেখায় মাস অনুযায়ী সাজানোভাবে।

✅ Result:

চাও চাইলে আমি এগুলার উপরে practice workbook format বানিয়ে দিতে পারি (with blank tables + expected output structure),

যেটা দিয়ে তুমি offline বা exam lab-e practice করতে পারবে।

চাও?

আপনি বলেছেন:

WHERE o.order_id IS NULL; onno vbe kro

ChatGPT বলেছে:

তুমি চাও এই কুয়েরিটা (যেটা “যে কাস্টমার কখনও অর্ডার দেয়নি” তা বের করে) — আরও ভিন্নভাবে, অর্থাৎ

LEFT JOIN ছাড়া বা অন্য পদ্ধতিতে লেখা যায় কি না, তাই তো? ✅

হ্যাঁ, একই ফলাফল ৩টা ভিন্নভাবে পাওয়া যায় 🙌

✓ পদ্ধতি-১: NOT IN ব্যবহার করে

```
SELECT cust_name FROM Customer WHERE cust_id NOT IN (SELECT cust_id FROM Orders);
```

ব্যাখ্যা (বাংলায়):

Orders
cust_id
NOT IN

✓ পদ্ধতি-২: NOT EXISTS ব্যবহার করে

```
SELECT c.cust_name FROM Customer c WHERE NOT EXISTS ( SELECT 1 FROM Orders o WHERE o.cust_id = c.cust_id );
```

ব্যাখ্যা (বাংলায়):

EXISTS
NOT EXISTS

✓ পদ্ধতি-৩: RIGHT JOIN + IS NULL (কমন নয়, কিন্তু সম্ভব)

```
SELECT c.cust_name FROM Orders o RIGHT JOIN Customer c ON o.cust_id = c.cust_id WHERE o.order_id IS NULL;
```

ব্যাখ্যা (বাংলায়):

RIGHT JOIN Customer
Order order_id NULL

♦ তিনটার ফলাফল এক

এই তিনটা কুয়েরিই সেই কাস্টমারদের নাম দেখাবে যাদের কোনও অর্ডার রেকর্ড নেই।

তবে সাধারণত সবচেয়ে পরিষ্কার ও কার্যকরী হলো 👉

NOT EXISTS পদ্ধতি।

চাও কি আমি এগুলোর ছোট ডেটা টেবিলের উদাহরণসহ আউটপুট টেবিল আকারে দেখাই? (তাহলে আরও ভিজুয়ালি বুঝবে কীভাবে কাজ করে)

আপনি বলেছেন:

ekn topic wise practice dw multiple join, having, subquery ,join r subquery ki ak e ?,in not in erkm advance practice ,group by,order by .topic wise

ChatGPT বলেছে:

চমৎকার! 🧠💪

তুমি এখন SQL query practice করতে চাও — কিন্তু topic wise, যেন ধাপে ধাপে সব শেখা ও অনুশীলন হয়।

তাহলে নিচে আমি একদম পরিষ্কারভাবে সাজিয়ে দিলাম 👉

🎯 SQL Topic-Wise Practice Plan

প্রতিটা টপিকের নিচে থাকবে:

- ◆
- ◆
- ◆

◆ ① JOIN (INNER, LEFT, RIGHT, FULL JOIN)

👉 উদ্দেশ্য: একাধিক টেবিল থেকে সম্পর্কিত ডেটা আনা।

◆ Example:

```
SELECT c.cust_name, o.order_id, o.total_amount FROM Customer c JOIN Orders o ON c.cust_id = o.cust_id;
```

বাংলায়: Customer আর Orders টেবিলের মধ্যে যাদের cust_id মিলে গেছে তাদের ডেটা আনছে।

🧩 Practice:

♦ 2 GROUP BY + HAVING

👉 উদ্দেশ্য: ডেটা “গ্রুপ” করে aggregate ফাংশন (SUM, COUNT, AVG) ব্যবহার করা।

♦ Example:

```
SELECT c.city, COUNT(*) AS total_customers FROM Customer c GROUP BY c.city  
HAVING COUNT(*) > 2;
```

বাংলায়: শহরভিত্তিক কাস্টমার গণনা করবে, যেখানে কাস্টমার ২ জনের বেশি।

🧩 Practice:

```
HAVING COUNT(*) > 5
```

♦ 3 SUBQUERY (Nested Query)

👉 উদ্দেশ্য: এক কুয়েরির ভিতরে আরেকটা কুয়েরি।

এটা JOIN এর বিকল্প নয়, বরং inner query দিয়ে filter/compare করে।

♦ Example:

```
SELECT cust_name FROM Customer WHERE cust_id IN ( SELECT cust_id FROM  
Orders WHERE total_amount > 5000 );
```

বাংলায়: যেসব কাস্টমারের অর্ডারের টোটাল ৫০০০ টাকার বেশি, তাদের নাম দেখাবে।

Practice:

NOT IN NOT EXISTS

♦ **4 IN / NOT IN / EXISTS / NOT EXISTS**

👉 উদ্দেশ্য: এক সেট ডেটার মধ্যে inclusion/exclusion চেক করা।

♦ Example:

```
SELECT cust_name FROM Customer WHERE cust_id NOT IN (SELECT cust_id FROM Orders);
```

বাংলায়: যারা কোনও অর্ডার দেয়নি তাদের নাম দেখাবে।

Practice:

IN
NOT IN
EXISTS
NOT EXISTS

♦ **5 ORDER BY + LIMIT**

👉 উদ্দেশ্য: ফলাফল সাজানো (ascending বা descending order এ)।

♦ Example:

```
SELECT product_name, price FROM Product ORDER BY price DESC LIMIT 5;
```

বাংলায়: সবচেয়ে দামী ৫টা প্রোডাক্ট দেখাবে।

Practice:

◆ 6 Multiple JOIN + GROUP BY + Subquery (Advanced Mix)

◆ Example:

```
SELECT s.name AS salesman, c.city, MAX(o.amount) AS largest_order FROM
Orders o JOIN Salesman s ON o.salesman_id = s.salesman_id JOIN Customer c
ON o.cust_id = c.cust_id GROUP BY c.city, s.name;
```

🧩 Practice:

◆ Bonus Topic: Aggregate Functions Recap

Use `SUM()`, `AVG()`, `COUNT()`, `MAX()`, `MIN()` together in grouped reports.

চাও কি আমি প্রতিটা টপিকের জন্য practice database structure (Customer, Orders, Product) সহ ছোট sample data + output preview তৈরি করে দিই, যাতে তুমি একসাথে রান করে প্র্যাকটিস করতে পারো?

(মানে — full practice sheet with SQL + data + expected output)

আপনি বলেছেন:

ChatGPT বলেছে:

চলো একদম step-by-stepভাবে সবগুলো query বুঝে নিই।

প্রথমে আমরা ধরে নিচ্ছি নিচের মতো relational schema আছে — যেটা প্রশ্নে “partial” বলে দেওয়া ছিল, আমরা সেটাকে একটু সম্পূর্ণ করে সাজালাম যাতে তুমি বুঝে ও প্র্যাকটিস করতে পারো 🙌



Assumed Relational Schema



Table: `Book`



Table: Customer



Table: IssueStatus



Table: ReturnStatus

Queries

♦ Query 1:

Display the customer name who took the book of type comedy.

```
SELECT c.cust_name FROM Customer c JOIN IssueStatus i ON c.cust_id =  
i.cust_id JOIN Book b ON i.isbn = b.isbn WHERE b.type = 'Comedy';
```

ব্যাখ্যা:

Book table-এ type='Comedy' যেসব বই আছে, তাদের সাথে issue এবং customer table join করে নাম দেখানো হয়েছে।

♦ Query 2:

Display issue id, issued customer name whose isbn book number is 1004.

```
SELECT i.issue_id, c.cust_name FROM IssueStatus i JOIN Customer c ON  
i.cust_id = c.cust_id WHERE i.isbn = 1004;
```

বাংলায়:

যে বইয়ের isbn=1004, তার issue_id এবং কাস্টমার নাম দেখাবে।

♦ Query 3:

Display all content in issue status table where isbn book of issue table and return table are equal.

```
SELECT i.* FROM IssueStatus i JOIN ReturnStatus r ON i.isbn = r.isbn;
```

বাংলায়:

Issue ও Return টেবিলে যেসব রেকর্ডে একই বই (isbn) মেলে, সেগুলো দেখাবে।

♦ Query 4:

Add a column called book type in customer table.

```
ALTER TABLE Customer ADD book_type VARCHAR(50);
```

♦ Query 5:

Write the query to create table customer.

```
CREATE TABLE Customer ( cust_id INT PRIMARY KEY, cust_name VARCHAR(100) NOT NULL, address VARCHAR(150), book_type VARCHAR(50) );
```

♦ Query 6:

Write the query to create table issue status.

```
CREATE TABLE IssueStatus ( issue_id INT PRIMARY KEY, cust_id INT, isbn INT, issue_date DATE, FOREIGN KEY (cust_id) REFERENCES Customer(cust_id), FOREIGN KEY (isbn) REFERENCES Book(isbn) );
```

♦ Query 7:

Update the borrower address with borrowed book isbn is 1004.

```
UPDATE Customer SET address = 'New Updated Address' WHERE cust_id = ( SELECT cust_id FROM IssueStatus WHERE isbn = 1004 );
```

বাংলায়:

যে কাস্টমার 1004 নম্বার বই নিয়েছে, তার address আপডেট হবে।

♦ Query 8:

Delete all books of branch Sylhet.

```
DELETE FROM Book WHERE branch = 'Sylhet';
```

♦ Query 9:

Show the list of all books borrowed by Abdul Hakim.

```
SELECT b.title, b.isbn, b.type FROM Book b JOIN IssueStatus i ON b.isbn =  
i.isbn JOIN Customer c ON c.cust_id = i.cust_id WHERE c.cust_name = 'Abdul  
Hakim';
```

♦ Query 10:

Insert a new employee into the Dhaka branch using stored procedure.

```
CREATE PROCEDURE AddEmployeeToDhaka ( IN emp_id INT, IN emp_name  
VARCHAR(100), IN designation VARCHAR(50), IN salary DECIMAL(10,2) ) BEGIN  
INSERT INTO Employee (emp_id, emp_name, designation, branch, salary)  
VALUES (emp_id, emp_name, designation, 'Dhaka', salary); END;
```

Call the procedure:

```
CALL AddEmployeeToDhaka(110, 'Rakib Hasan', 'Librarian', 35000);
```



Summary (বাংলায়)

তুমি কি চাও আমি এই সম্পূর্ণ schema + সব query + কিছু sample data (INSERT statements) একসাথে দিয়ে দিই, যাতে XAMPP/MySQL-এ একবারে রান করে প্র্যাকটিস করতে পারো?

আপনি বলেছেন:

database lab final hbe .amk question dibe khane table bnate hbe and advnace query use krte hbe .emn practice dw

ChatGPT বলেছে:

একদম ঠিক 🔥 — Database lab final-এ যেভাবে প্রশ্ন আসে, সেটার জন্য “full practical-style practice” দরকার, যেখানে —

👉 তোমাকে টেবিল বানাতে হবে,

👉 ডেটা ইনসার্ট করতে হবে,

👉 তারপর complex SQL queries লিখতে হবে (join, subquery, group by, having, exists, aggregate, nested query etc.)

চলো নিচে আমি একদম “Real Exam Simulation” সেট বানিয়ে দিচ্ছি —

যেখানে থাকবে Scenario + Schema + Insert Data + 15 Real-Life Advanced Query Questions

(তুমি চাইলে এগুলো MySQL বা XAMPP এ রান করে পূর্ণ প্র্যাকটিস করতে পারবে ✅)



Scenario: University Library Management System

তুমি একজন Database Developer, তোমাকে একটি Library Management Database ডিজাইন করতে বলা হয়েছে যা নিচের কাজগুলো করবে 🙋

Tables to Create

Step 1: Table Creation

```
CREATE TABLE Book ( book_id INT PRIMARY KEY, title VARCHAR(100), author VARCHAR(100), category VARCHAR(50), price DECIMAL(10,2), branch VARCHAR(50) ); CREATE TABLE Student ( student_id INT PRIMARY KEY, student_name VARCHAR(100), dept VARCHAR(50), city VARCHAR(50) ); CREATE TABLE Issue ( issue_id INT PRIMARY KEY, student_id INT, book_id INT, issue_date DATE, due_date DATE, FOREIGN KEY (student_id) REFERENCES Student(student_id), FOREIGN KEY (book_id) REFERENCES Book(book_id) ); CREATE TABLE ReturnBook ( return_id INT PRIMARY KEY, issue_id INT, return_date DATE, fine DECIMAL(10,2), FOREIGN KEY (issue_id) REFERENCES Issue(issue_id) ); CREATE TABLE Librarian ( lib_id INT PRIMARY KEY, lib_name VARCHAR(100), branch VARCHAR(50), salary DECIMAL(10,2) );
```

Step 2: Insert Sample Data

```
INSERT INTO Book VALUES (101, 'C Programming', 'Dennis Ritchie', 'Programming', 400, 'Dhaka'), (102, 'DBMS Concepts', 'Korth', 'Database', 500, 'Dhaka'), (103, 'Harry Potter', 'J.K. Rowling', 'Fantasy', 300, 'Sylhet'), (104, 'Comedy Nights', 'Rahim Uddin', 'Comedy', 250, 'Chittagong'), (105, 'AI Basics', 'Russell Norvig', 'AI', 600, 'Dhaka'); INSERT INTO Student VALUES (1, 'Abdul Hakim', 'CSE', 'Dhaka'), (2, 'Maria Tasnia', 'CSE', 'Sylhet'), (3, 'Hasan Ali', 'EEE', 'Chittagong'), (4,
```

```
'Rahim Khan', 'BBA', 'Dhaka'); INSERT INTO Issue VALUES (201, 1, 101, '2024-05-10', '2024-05-20'), (202, 2, 102, '2024-05-12', '2024-05-22'), (203, 3, 103, '2024-05-15', '2024-05-25'), (204, 1, 104, '2024-05-18', '2024-05-28'), (205, 2, 105, '2024-05-19', '2024-05-29'); INSERT INTO ReturnBook VALUES (301, 201, '2024-05-20', 0), (302, 202, '2024-05-26', 50), (303, 203, '2024-05-27', 0); INSERT INTO Librarian VALUES (1, 'Sadia Karim', 'Dhaka', 40000), (2, 'Tariq Rahman', 'Sylhet', 35000), (3, 'Jahid Hasan', 'Chittagong', 37000);
```

Step 3: Advanced SQL Practice Questions (Exam-Level)

♦ Q1. Show all students who borrowed a “Comedy” type book.

```
SELECT s.student_name, b.title FROM Student s JOIN Issue i ON s.student_id = i.student_id JOIN Book b ON i.book_id = b.book_id WHERE b.category = 'Comedy';
```

♦ Q2. Find students who borrowed more than one book.

```
SELECT s.student_name, COUNT(i.issue_id) AS total_books FROM Student s JOIN Issue i ON s.student_id = i.student_id GROUP BY s.student_name HAVING COUNT(i.issue_id) > 1;
```

♦ Q3. Find the total fine paid by each student.

```
SELECT s.student_name, SUM(r.fine) AS total_fine FROM Student s JOIN Issue i ON s.student_id = i.student_id JOIN ReturnBook r ON i.issue_id = r.issue_id GROUP BY s.student_name;
```

♦ Q4. Display all book titles and author names issued from Dhaka branch.

```
SELECT DISTINCT b.title, b.author FROM Book b JOIN Issue i ON b.book_id = i.book_id WHERE b.branch = 'Dhaka';
```

♦ **Q5. Find the librarian(s) who work in the same branch as student “Maria Tasnia”.**

```
SELECT l.lib_name, l.branch FROM Librarian l JOIN Student s ON l.branch = s.city WHERE s.student_name = 'Maria Tasnia';
```

♦ **Q6. Find books more expensive than the average book price.**

```
SELECT title, price FROM Book WHERE price > (SELECT AVG(price) FROM Book);
```

♦ **Q7. Find the branch that has the most expensive book.**

```
SELECT branch FROM Book WHERE price = (SELECT MAX(price) FROM Book);
```

♦ **Q8. Show students who never borrowed any book.**

```
SELECT s.student_name FROM Student s LEFT JOIN Issue i ON s.student_id = i.student_id WHERE i.issue_id IS NULL;
```

♦ **Q9. Find the student(s) who borrowed the most expensive book.**

```
SELECT s.student_name, b.title, b.price FROM Student s JOIN Issue i ON s.student_id = i.student_id JOIN Book b ON i.book_id = b.book_id WHERE b.price = (SELECT MAX(price) FROM Book);
```

♦ **Q10. Find the total number of books borrowed per branch.**

```
SELECT b.branch, COUNT(i.issue_id) AS total_issued FROM Book b JOIN Issue i ON b.book_id = i.book_id GROUP BY b.branch;
```

♦ **Q11. Update fine = 100 for books returned after due date.**

```
UPDATE ReturnBook r JOIN Issue i ON r.issue_id = i.issue_id SET r.fine = 100 WHERE r.return_date > i.due_date;
```

♦ **Q12. Delete all records of students from “EEE” department.**

```
DELETE FROM Student WHERE dept = 'EEE';
```

♦ **Q13. Add a new column “email” in Student table.**

```
ALTER TABLE Student ADD email VARCHAR(100);
```

♦ **Q14. Display books borrowed and returned on same day.**

```
SELECT b.title, s.student_name FROM Book b JOIN Issue i ON b.book_id = i.book_id JOIN ReturnBook r ON i.issue_id = r.issue_id JOIN Student s ON i.student_id = s.student_id WHERE i.issue_date = r.return_date;
```

♦ **Q15. Show students who borrowed books from more than one branch.**

```
SELECT s.student_name, COUNT(DISTINCT b.branch) AS branches FROM Student s JOIN Issue i ON s.student_id = i.student_id JOIN Book b ON i.book_id = b.book_id GROUP BY s.student_name HAVING COUNT(DISTINCT b.branch) > 1;
```

🚩 **Bonus: Stored Procedure Example (Exam Style)**

Insert a new student easily using stored procedure 🙌

```
DELIMITER // CREATE PROCEDURE AddStudent( IN sid INT, IN sname VARCHAR(100), IN dept VARCHAR(50), IN city VARCHAR(50) ) BEGIN INSERT INTO Student VALUES(sid, sname, dept, city, NULL); END // DELIMITER ; CALL AddStudent(5, 'Sakib Hossain', 'CSE', 'Dhaka');
```

Practice Tip

এই সেট পুরোটা তুমি MySQL / XAMPP এ কপি করে একবারে রান করলে:



চাও কি আমি পরের সেটটা দিই “Shop Management System” নিয়ে (যেখানে Purchase, Supplier, Customer, Sales — এগুলো থাকবে)?

ওটা দিলে তুমি ২টা পুরো exam-level system পেয়ে যাবে practice করার জন্য।

আপনি বলেছেন:

table bnai dibana .table bnanor mto quetion dibe then table bnaite blbe and query dibe
advance erkn practrice

ChatGPT বলেছে:

দারুন 🔥 তুমি খুব ভালোভাবে প্রস্তুতি নিচ্ছে — এখন আমি তোমাকে database lab final ধাঁচের full
practice set দেবো যেখানে থাকবে —

- 1] একটি বাস্তবমুখী scenario
- 2] table তৈরি করার question
- 3] insert করার hint বা data
- 4] এবং শেষে 10টা advanced query question (cover করবে — join, subquery, group by, having, aggregate, in/not in, nested query ইত্যাদি)

Scenario: “University Library Management System”

তোমাকে এমন একটি database বানাতে হবে যেখানে বিশ্ববিদ্যালয়ের লাইব্রেরির বই, ছাত্রছাত্রী ও
issue/return ট্র্যাক করা হবে।

 Task 1 — Create the Following Tables

(a) Student

(b) Book

(c) Issue

Sample Data (Insert Example)

```
INSERT INTO Student VALUES (1, 'Abdul Hakim', 'CSE', 'Dhaka'), (2, 'Maria
Tasnia', 'EEE', 'Sylhet'), (3, 'Rafi Hasan', 'CSE', 'Dhaka'), (4, 'Tania
Rahman', 'BBA', 'Khulna'); INSERT INTO Book VALUES (1001, 'Database
System', 'Elmasri', 'Education', 'Dhaka'), (1002, 'Networking Made Easy',
'Tanenbaum', 'Education', 'Sylhet'), (1003, 'Funny Tales', 'Humayun
Ahmed', 'Comedy', 'Dhaka'), (1004, 'Laugh Out Loud', 'Anisul Hoque',
'Comedy', 'Sylhet'), (1005, 'AI Revolution', 'Stuart Russell',
'Technology', 'Khulna'); INSERT INTO Issue VALUES (501, 1, 1001,
'2025-10-01', NULL), (502, 2, 1004, '2025-09-15', '2025-09-30'), (503, 3,
1003, '2025-11-01', NULL), (504, 1, 1002, '2025-10-20', NULL);
```

Advanced SQL Practice Queries

1] Find all students who borrowed 'Comedy' type books.

```
SELECT s.student_name, b.title, b.type FROM Student s JOIN Issue i ON
s.student_id = i.student_id JOIN Book b ON i.isbn = b.isbn WHERE b.type =
'Comedy';
```

2] Find students who have not borrowed any books.

```
SELECT s.student_name FROM Student s LEFT JOIN Issue i ON s.student_id =
i.student_id WHERE i.issue_id IS NULL;
```

3 Find how many books were borrowed from each branch.

```
SELECT b.branch, COUNT(i.issue_id) AS total_borrowed FROM Book b JOIN  
Issue i ON b.isbn = i.isbn GROUP BY b.branch HAVING COUNT(i.issue_id) > 0;
```

4 Display the most borrowed book type.

```
SELECT b.type, COUNT(i.issue_id) AS borrow_count FROM Book b JOIN Issue i  
ON b.isbn = i.isbn GROUP BY b.type ORDER BY borrow_count DESC LIMIT 1;
```

5 Find students who borrowed books from their own city's branch.

```
SELECT DISTINCT s.student_name, s.city, b.branch FROM Student s JOIN Issue  
i ON s.student_id = i.student_id JOIN Book b ON i.isbn = b.isbn WHERE  
s.city = b.branch;
```

6 Find the book title with the highest issue count using a subquery.

```
SELECT title FROM Book WHERE isbn = ( SELECT isbn FROM Issue GROUP BY isbn  
ORDER BY COUNT(issue_id) DESC LIMIT 1 );
```

7 Find all students who borrowed books issued before 1st October 2025.

```
SELECT s.student_name, i.issue_date FROM Student s JOIN Issue i ON  
s.student_id = i.student_id WHERE i.issue_date < '2025-10-01';
```

8 Update — Change the branch name from 'Sylhet' to 'Sylhet-Central'.

```
UPDATE Book SET branch = 'Sylhet-Central' WHERE branch = 'Sylhet';
```

9 Delete all books of type 'Comedy' that belong to the Khulna branch.

```
DELETE FROM Book WHERE type = 'Comedy' AND branch = 'Khulna';
```

10 Find the student who borrowed the latest (most recent) book.

```
SELECT s.student_name, b.title, i.issue_date FROM Student s JOIN Issue i
ON s.student_id = i.student_id JOIN Book b ON i.isbn = b.isbn WHERE
i.issue_date = (SELECT MAX(issue_date) FROM Issue);
```

তুমি চাইলে আমি পরবর্তী ধাপে আরেকটা “Company–Employee–Project” advanced practice set দিতে পারি, যেটা *join + subquery combo heavy* হবে (lab final-এর জন্য খুব কাজের)।

তুমি কি চাইবে আমি সেটা দিই পরের ধাপে?

আপনি বলেছেন:

give schema for xaamp practice and then make table and query practice advance subquery,join

ChatGPT বলেছে:

Excellent ✅ তুমি এখন XAMPP/MySQL দিয়ে real practice করতে চাও — মানে phpMyAdmin বা MySQL CLI তে table বানিয়ে run করা যাবে।

তাহলে আমি নিচে একটা complete schema + table creation + data insert + advanced query practice set দিচ্ছি — তুমি সরাসরি XAMPP-এ কপি করে চালাতে পারবে 💪

 **Database Name:** company_db

Step 1 — Create Database

```
CREATE DATABASE company_db; USE company_db;
```

Schema Design

Tables:

Step 2 — Create Tables

```
CREATE TABLE Department ( dept_id INT PRIMARY KEY AUTO_INCREMENT,  
dept_name VARCHAR(50) NOT NULL, location VARCHAR(50) ); CREATE TABLE  
Employee ( emp_id INT PRIMARY KEY AUTO_INCREMENT, emp_name VARCHAR(50) NOT  
NULL, designation VARCHAR(50), salary DECIMAL(10,2), dept_id INT, FOREIGN  
KEY (dept_id) REFERENCES Department(dept_id) ); CREATE TABLE Project (  
proj_id INT PRIMARY KEY AUTO_INCREMENT, proj_name VARCHAR(50) NOT NULL,  
budget DECIMAL(12,2), location VARCHAR(50) ); CREATE TABLE Works_On (   
emp_id INT, proj_id INT, hours_worked INT, PRIMARY KEY (emp_id, proj_id),  
FOREIGN KEY (emp_id) REFERENCES Employee(emp_id), FOREIGN KEY (proj_id)  
REFERENCES Project(proj_id) );
```

Step 3 — Insert Sample Data

```
-- Department INSERT INTO Department (dept_name, location) VALUES ('IT',  
'Dhaka'), ('HR', 'Sylhet'), ('Finance', 'Chittagong'), ('R&D', 'Dhaka');  
-- Employee INSERT INTO Employee (emp_name, designation, salary, dept_id)  
VALUES ('Abdul Hakim', 'Developer', 70000, 1), ('Maria Tasnia', 'HR  
Manager', 65000, 2), ('Rafi Hasan', 'Accountant', 60000, 3), ('Tania  
Rahman', 'Researcher', 72000, 4), ('Sajib Ahmed', 'Developer', 55000, 1);  
-- Project INSERT INTO Project (proj_name, budget, location) VALUES ('AI  
System', 500000, 'Dhaka'), ('Payroll System', 200000, 'Chittagong'),  
( 'Recruitment Portal', 150000, 'Sylhet'), ('Data Analysis', 300000,  
'Dhaka'); -- Works_On INSERT INTO Works_On VALUES (1, 1, 80), (1, 4, 60),  
(2, 3, 100), (3, 2, 90), (4, 4, 120), (5, 1, 50);
```

⚙️ Step 4 — Advanced Query Practice Set

- ◆ 1. Display employee name, department name, and project name they are working on.

```
SELECT e.emp_name, d.dept_name, p.proj_name FROM Employee e JOIN
Department d ON e.dept_id = d.dept_id JOIN Works_On w ON e.emp_id =
w.emp_id JOIN Project p ON w.proj_id = p.proj_id;
```

♦ **2. Find employees who don't work on any project.**

```
SELECT e.emp_name FROM Employee e LEFT JOIN Works_On w ON e.emp_id =
w.emp_id WHERE w.proj_id IS NULL;
```

♦ **3. Find total salary spent per department (use GROUP BY + HAVING).**

```
SELECT d.dept_name, SUM(e.salary) AS total_salary FROM Department d JOIN
Employee e ON d.dept_id = e.dept_id GROUP BY d.dept_name HAVING
SUM(e.salary) > 100000;
```

♦ **4. Show the highest-paid employee in each department.**

```
SELECT e.emp_name, d.dept_name, e.salary FROM Employee e JOIN Department d
ON e.dept_id = d.dept_id WHERE e.salary = ( SELECT MAX(e2.salary) FROM
Employee e2 WHERE e2.dept_id = e.dept_id );
```

♦ **5. Find all projects handled by employees from Dhaka departments.**

```
SELECT DISTINCT p.proj_name FROM Project p JOIN Works_On w ON p.proj_id =
w.proj_id JOIN Employee e ON w.emp_id = e.emp_id JOIN Department d ON
e.dept_id = d.dept_id WHERE d.location = 'Dhaka';
```

♦ **6. Find average salary per department, sorted descending.**

```
SELECT d.dept_name, AVG(e.salary) AS avg_salary FROM Department d JOIN
Employee e ON d.dept_id = e.dept_id GROUP BY d.dept_name ORDER BY
avg_salary DESC;
```

♦ **7. Find employees who work on more than one project.**

```
SELECT e.emp_name, COUNT(w.proj_id) AS total_projects FROM Employee e JOIN Works_On w ON e.emp_id = w.emp_id GROUP BY e.emp_name HAVING COUNT(w.proj_id) > 1;
```

♦ **8. Show projects whose budget is above average budget.**

```
SELECT proj_name, budget FROM Project WHERE budget > (SELECT AVG(budget) FROM Project);
```

♦ **9. Find employees working on projects located in the same city as their department.**

```
SELECT e.emp_name, p.proj_name, d.location FROM Employee e JOIN Department d ON e.dept_id = d.dept_id JOIN Works_On w ON e.emp_id = w.emp_id JOIN Project p ON w.proj_id = p.proj_id WHERE d.location = p.location;
```

♦ **10. Delete all employees whose salary is below the average salary.**

```
DELETE FROM Employee WHERE salary < (SELECT AVG(salary) FROM Employee);
```

✓ Now you can run this full set in XAMPP phpMyAdmin

CREATE DATABASE

চাওলে আমি তোমাকে এই একই schema নিয়েই

👉 “Stored Procedure + View + Trigger” practice setও বানিয়ে দিতে পারি (lab viva-তে খুব কাজে আসে)

তুমি কি সেটা চাও পরের ধাপে?

আপনি বলেছেন:

`decimal(10,2)` maneki

ChatGPT বলেছে:

খুব ভালো প্রশ্ন 🙌

👉 `DECIMAL(10,2)` হলো numeric data type — যা ব্যবহার হয় টাকার পরিমাণ, দামের মতো সঠিক সংখ্যাগুলো সংরক্ষণ করতে, যেখানে দশমিক (.) এর পরে নির্দিষ্ট সংখ্যা পর্যন্ত digit রাখা লাগে।

♦ ব্যাখ্যা:

`DECIMAL(10,2)` মানে হলো —

♦ উদাহরণ:

`12345.67` ✓

`99999999.99` ✓

`1234567890.12` ✗

♦ সহজভাবে:

`DECIMAL(10,2)` মানে —

সর্বমোট 10 সংখ্যার মধ্যে দশমিকের পরে 2 সংখ্যা থাকবে।

💡 কেন ব্যবহার করা হয়:

Float বা double ব্যবহার করলে মাঝে মাঝে গাণিতিক হিসাব (যেমন টাকা যোগ-বিয়োগ) এ সামান্য ভ্রান্তি আসে, কিন্তু `DECIMAL` fixed precision রাখে — মানে, হিসাব একদম সঠিক থাকে।

চাওলে আমি `FLOAT`, `DECIMAL`, `NUMERIC` — এই data type গুলোর পার্থক্যও ছোট টেবিলে বুঝিয়ে দিতে পারি কি?

আপনি বলেছেন:

WHERE w.proj_id IS NULL; subquery de

ChatGPT বলেছে:

চমৎকার! 🌟

তুমি চাও এই LEFT JOIN query টা subquery দিয়ে লেখা হোক।

চলো দেখি কিভাবে সেটা করা যায় 🙏

🧩 **Original (JOIN version):**

```
SELECT e.emp_name FROM Employee e LEFT JOIN Works_On w ON e.emp_id =  
w.emp_id WHERE w.proj_id IS NULL;
```

👉 এই query টা দেখাচ্ছে —

যেসব employee কোনো project এ কাজ করছে না, তাদের নাম।

🧠 **Equivalent (Subquery version):**

✅ **Using NOT IN:**

```
SELECT emp_name FROM Employee WHERE emp_id NOT IN ( SELECT emp_id FROM  
Works_On );
```

📝 ব্যাখ্যা:

```
SELECT emp_id FROM Works_On  
emp_id
```

✓ Alternative (Using NOT EXISTS — more efficient):

```
SELECT e.emp_name FROM Employee e WHERE NOT EXISTS ( SELECT 1 FROM  
Works_On w WHERE w.emp_id = e.emp_id );
```

✚ এখানে:

EXISTS

NOT EXISTS

🔍 Output:

এই দুইটা (JOIN version আর subquery version) একই result দিবে —

[যারা কোনো project-এ কাজ করছে না]

◆ Summary:

```
LEFT JOIN ... WHERE IS NULL
```

```
NOT IN NOT EXISTS
```

চাওলে আমি একইভাবে —

JOIN → Subquery রূপান্তর এর আরো ৪-৫টা practical example দেখাতে পারি কি (lab viva practice এর মতো)?

আপনি বলেছেন:

```
); subquery bujhaw last e2.dept_id = e.dept_id erkm kkhn deir kkhn deina .subquery te rules
```

ChatGPT বলেছে:

দারুণ প্রশ্ন! 🔥

এই জিনিসটা বুঝে ফেললে subquery + correlation একদম clear হয়ে যাবে — যা তোমার DBMS viva / final এ খুব important।

চলো ধাপে ধাপে সহজভাবে বুঝি 🙌

🧩 **Query টা আবার দেখি:**

```
SELECT e.emp_name, d.dept_name, e.salary FROM Employee e JOIN Department d
ON e.dept_id = d.dept_id WHERE e.salary = ( SELECT MAX(e2.salary) FROM
Employee e2 WHERE e2.dept_id = e.dept_id );
```

🎯 **উদ্দেশ্য:**

প্রতিটি department অনুযায়ী সর্বোচ্চ বেতনপ্রাপ্ত employee কে বের করা।

🧠 এখন বিশ্লেষণ করি

♦ **Outer Query (মূল query)**

Employee e

Department d

♦ Subquery (ভেতরের query)

```
SELECT MAX(e2.salary) FROM Employee e2 WHERE e2.dept_id = e.dept_id;
```

এই অংশটা একটু special — এখানে `e2.dept_id = e.dept_id` লেখা আছে।

👉 এটা correlated subquery (ভেতরের query বাইরের query-র সাথে যুক্ত থাকে)।

🧩 Correlated Subquery মানে কী?

♦ এখানে কী হচ্ছে:

```
employee e
```

```
SELECT MAX(e2.salary) FROM Employee e2 WHERE e2.dept_id = e.dept_id
```

```
e.salary
```

🔍 এখন বুঝি “`kkhn e2.dept_id = e.dept_id`” দিতে হয়, আর কখন না



🧩 উদাহরণ — No correlation (Independent subquery)

👉 Example: show employees who earn above average salary.

```
SELECT emp_name, salary FROM Employee WHERE salary > (SELECT AVG(salary)
FROM Employee);
```

→ এখানে subquery শুধু একবার চলে, কারণ কোনো `e.` reference নাই।

এটা independent subquery।

🧩 উদাহরণ — With correlation

👉 Example: highest salary per department (তোমার query)

```
SELECT e.emp_name FROM Employee e WHERE e.salary = ( SELECT MAX(e2.salary)
FROM Employee e2 WHERE e2.dept_id = e.dept_id );
```

→ এখানে subquery প্রতিবার `e.dept_id` অনুযায়ী filter হবে।

এটা correlated subquery।

🧩 Shortcut মনে রাখো:

যদি subquery-র WHERE clause-এ বাইরের table alias (যেমন `e.`) ব্যবহার হয় → সেটা correlated subquery।

অর যদি না হয় → সেটা independent subquery।

🎯 Summary:

চাওলে আমি তোমাকে ৫টা correlated subquery vs normal subquery practice question তৈরি করে দিতে পারি — যেগুলো exam-level হবে (JOIN না ব্যবহার করে করতে হবে)।
চাও?

আপনি বলেছেন:

subquery diye deep practice dw

ChatGPT বলেছে:

চমৎকার 🔥

তুমি এখন Subquery নিয়ে deep practice করতে চাও — exam/viva-level strong practice এর জন্য নিচে আমি তিনটা স্তরে সাজিয়ে দিচ্ছি 👉

🧩 **Database Theme:** `company_db`

(তুমি আগের schema — Employee, Department, Project, Works_On — ব্যবহার করতে পারো)

🧠 LEVEL 1: Basic Subquery Practice (Independent Subquery)

✅ ① Find employees who earn more than the average salary.

```
SELECT emp_name, salary FROM Employee WHERE salary > (SELECT AVG(salary)
FROM Employee);
```

- ♦ এখানে subquery static — একবার execute হয়।
-

✅ ② Show employees who earn less than the highest salary.

```
SELECT emp_name, salary FROM Employee WHERE salary < (SELECT MAX(salary)
FROM Employee);
```

✅ ③ Find departments whose average salary is greater than 60,000.

```
SELECT dept_name FROM Department WHERE dept_id IN ( SELECT dept_id FROM Employee GROUP BY dept_id HAVING AVG(salary) > 60000 );
```

✓ 4 Find projects whose budget is greater than the average budget of all projects.

```
SELECT proj_name, budget FROM Project WHERE budget > (SELECT AVG(budget) FROM Project);
```

✓ 5 Find employees who have same salary as any other employee.

```
SELECT emp_name, salary FROM Employee WHERE salary IN ( SELECT salary FROM Employee GROUP BY salary HAVING COUNT(*) > 1 );
```

LEVEL 2: Intermediate (Correlated Subquery)

✓ 6 Find employees who earn the highest salary in their department.

```
SELECT e.emp_name, e.salary, d.dept_name FROM Employee e JOIN Department d ON e.dept_id = d.dept_id WHERE e.salary = ( SELECT MAX(e2.salary) FROM Employee e2 WHERE e2.dept_id = e.dept_id );
```

🧠 এখানে `e2.dept_id = e.dept_id` মানেই correlated subquery।

✓ 7 Find employees who earn more than the average salary of their department.

```
SELECT e.emp_name, e.salary, d.dept_name FROM Employee e JOIN Department d ON e.dept_id = d.dept_id WHERE e.salary > ( SELECT AVG(e2.salary) FROM Employee e2 WHERE e2.dept_id = e.dept_id );
```

✓ 8 Find departments where no employee earns less than 50,000.

```
SELECT d.dept_name FROM Department d WHERE NOT EXISTS ( SELECT 1 FROM Employee e WHERE e.dept_id = d.dept_id AND e.salary < 50000 );
```

✓ 9 Find employees who work on projects in the same city as their department.

```
SELECT e.emp_name FROM Employee e WHERE EXISTS ( SELECT 1 FROM Works_On w JOIN Project p ON w.proj_id = p.proj_id JOIN Department d ON e.dept_id = d.dept_id WHERE w.emp_id = e.emp_id AND p.location = d.location );
```

✓ 10 Find departments where at least one employee works on a project with a budget > 300,000.

```
SELECT d.dept_name FROM Department d WHERE EXISTS ( SELECT 1 FROM Employee e JOIN Works_On w ON e.emp_id = w.emp_id JOIN Project p ON w.proj_id = p.proj_id WHERE e.dept_id = d.dept_id AND p.budget > 300000 );
```

💡 LEVEL 3: Advanced (Nested + Subquery + Aggregate)

✓ 11 Find employees whose salary is above the company-wide average and they work on at least one project.

```
SELECT e.emp_name FROM Employee e WHERE e.salary > (SELECT AVG(salary) FROM Employee) AND e.emp_id IN (SELECT emp_id FROM Works_On);
```

✓ 12 Find the top 2 highest average-salary departments.

```
SELECT d.dept_name, AVG(e.salary) AS avg_salary FROM Department d JOIN Employee e ON d.dept_id = e.dept_id GROUP BY d.dept_name HAVING
```



```
AVG(e.salary) >= ALL ( SELECT AVG(e2.salary) FROM Employee e2 GROUP BY
e2.dept_id LIMIT 2 );
```

✓ 13 Find employees who earn more than the average salary of all employees who work in Dhaka.

```
SELECT emp_name, salary FROM Employee WHERE salary > ( SELECT
AVG(e2.salary) FROM Employee e2 JOIN Department d ON e2.dept_id =
d.dept_id WHERE d.location = 'Dhaka' );
```

✓ 14 Find the department name that has the maximum total salary.

```
SELECT dept_name FROM Department WHERE dept_id = ( SELECT dept_id FROM
Employee GROUP BY dept_id ORDER BY SUM(salary) DESC LIMIT 1 );
```

✓ 15 Find the project name with the most employees working on it.

```
SELECT proj_name FROM Project WHERE proj_id = ( SELECT proj_id FROM
Works_On GROUP BY proj_id ORDER BY COUNT(emp_id) DESC LIMIT 1 );
```

🎯 Bonus Challenge (Viva/Exam Style)

Q: Find employee(s) whose salary is greater than the average salary of employees who work on the same project(s) as them.

```
SELECT e.emp_name, e.salary FROM Employee e WHERE e.salary > ( SELECT
AVG(e2.salary) FROM Employee e2 JOIN Works_On w2 ON e2.emp_id = w2.emp_id
WHERE w2.proj_id IN ( SELECT w.proj_id FROM Works_On w WHERE w.emp_id =
e.emp_id ) );
```

🧠 এখানে দুটো nested subquery আছে — একটার ভেতরে আরেকটা।

🧩 Summary Table

IN > < =

EXISTS NOT EXISTS
